NR2DT-03: Step 3 — Design

> **Series:** NR2DT — New Relic to Dynatrace Migration Steps | **Notebook:** 3 of 10 | **Created:** April 2026 | **Last Updated:** 10/31/2026

Overview

Goal of this step: lock in the target Dynatrace architecture before any artifact migrates. Bucket strategy, host group taxonomy, IAM model, and OpenPipeline routing are all decided here.

Procedural runbook — see **FAQ-01** (host group naming strategy) and **ORGNZ-02 / ORGNZ-99** (Grail bucket strategy) for the recommended design patterns this step uses.

Table of Contents

1. What You'll Produce
2. Bucket Design
3. Host Group Design
4. IAM Model
5. OpenPipeline Layout
6. Step Exit Criteria

Prerequisites

Requirement	Details
Audience	Migration lead + assigned engineer for this step

Requirement	Details
Completed	NR2DT-02 — Strategize
Format	Procedural step — use as a runbook; defer to NRLC for depth
Topic deep dives	FAQ-01 (host group naming), ORGNZ-02 / ORGNZ-99 (bucket strategy), IAM-04/05 (policy design), IAM-11 (WORKSHOP)

1. What You'll Produce

Artifact	Format	Purpose
`bucket-design.md`	Markdown	Bucket names, retention, pricing model
`host-group-design.md`	Markdown	Host group taxonomy and naming
`iam-model.md`	Markdown	Groups, policies, scopes
`openpipeline-design.md`	Markdown	Enrichment + routing + filter rules
`terraform/`	HCL	Phase 1 Terraform module skeleton

2. Bucket Design

Use the recommended pattern from **ORGNZ-02 — Understanding Grail Buckets** (with the consolidated reference in **ORGNZ-99**) as your starting template:

Bucket	Retention	Pricing	Purpose
`<org>_app_logs`	30 days	Retain w/ Included Queries	Application logs
`<org>_infra_logs`	14 days	Retain w/ Included Queries	Infrastructure / K8s
`<org>_security_logs`	365 days	Usage-based	Audit trail / PCI
`<org>_spans`	14 days	Usage-based	Distributed traces
`<org>_events`	90 days	Usage-based	Platform events
`<org>_bizevents`	90 days	Usage-based	Business events

Retain with Included Queries is a Log Management & Analytics option — *"Customers can split a log bucket's retention period into two parts"* — and covers only the first 10–35 days (*"Retain with Included Queries is available only for the first 10–35 days of retention."*). The DPS capabilities page lists no included-queries option for Traces or Events — only separate *Retain* and *Query* capabilities — so keep span, event and business-event buckets usage-based.

> **Sources:** Log Management & Analytics (DT docs), Retain with Included Queries (DT docs), DPS capabilities (DT docs).

Use `<org>_` as the prefix — it lets IAM policies scope with one `storage:bucket-name startsWith "<org>_"` condition.

Document any deviations from this template (e.g., per-app buckets) and why.

3. Host Group Design

Per **FAQ-01 — Host Group Naming Strategy**, group hosts by ownership boundary, not by team alone. Common dimensions:

- Environment (`prod-app` , `nonprod-app` , `prod-infra`)
- Compliance scope (`payment-pci`)
- Workload tier (`prod-batch` , `prod-stream`)

Plan a host group per (environment × ownership) combination. Avoid a single all-hosts group — it breaks per-environment thresholds, IAM scoping, and automation safety.

4. IAM Model

Recommended starting groups:

Group	Bucket Access
`<org>-admins`	All `<org>_` buckets
`<org>-sre`	All operational: app + infra logs, spans, events
`<org>-app-eng`	App logs only

Group	Bucket Access
`<org>-readonly`	All `<org>_` buckets except security

Each statement grants a Grail table permission (`storage:logs:read` and so on) together with `storage:buckets:read`, scoped by the `storage:bucket-name` condition. The bucket permission is needed as well as the table permission — the IAM reference describes `storage:buckets:read` as *"Required additionally to a table permission."*

```
// &lt;org&gt;-admins
ALLOW storage:buckets:read, storage:logs:read, storage:spans:read,
storage:events:read, storage:bizevents:read
  WHERE storage:bucket-name startsWith "&lt;org&gt;_";

// &lt;org&gt;-sre
ALLOW storage:buckets:read, storage:logs:read WHERE storage:bucket-name IN
("&lt;org&gt;_app_logs", "&lt;org&gt;_infra_logs");
ALLOW storage:buckets:read, storage:spans:read WHERE storage:bucket-name =
"&lt;org&gt;_spans";
ALLOW storage:buckets:read, storage:events:read WHERE storage:bucket-name =
"&lt;org&gt;_events";

// &lt;org&gt;-app-eng
ALLOW storage:buckets:read, storage:logs:read WHERE storage:bucket-name =
"&lt;org&gt;_app_logs";

// &lt;org&gt;-readonly — security bucket excluded from the ALLOW scope
ALLOW storage:buckets:read, storage:logs:read, storage:spans:read,
storage:events:read, storage:bizevents:read
  WHERE storage:bucket-name startsWith "&lt;org&gt;_" AND storage:bucket-name
!= "&lt;org&gt;_security_logs";
```

Do not exclude the security bucket with a conditional DENY on a table permission. It does not do what it reads as: *"Conditional DENY statements involving Grail table permissions are not supported and are executed as unconditional DENY."* A `DENY storage:logs:read WHERE storage:bucket-name = "<org>_security_logs";` bound to the SRE, app-eng or readonly group would remove **all** log access for that group. Exclude the bucket from the ALLOW scope instead, as `<org>-readonly` does above. If you do need a DENY, the only storage permission that accepts a conditional one is the bucket permission: *"Conditional DENY statements are supported only for the buckets:read permission."*

For policy design in depth (parameterized policies, boundaries), see **IAM-04** and **ORGNZ**.

> **Sources:** IAM policy statement syntax (DT docs), IAM policy reference (DT docs), read 10/05/2026.

5. OpenPipeline Layout

Routing rules in priority order (first match wins):

Priority	Matcher	Bucket
1	`matchesValue(log.source, "*audit*") OR loglevel == "CRITICAL"`	`<org>_security_logs`
2	`in(k8s.namespace.name, {"kube-system", "kube-public", ...})`	`<org>_infra_logs`
3	App namespace match	`<org>_app_logs`
4	catch-all	`default_logs` (review weekly)

Enrichment rules for any NR workload identifier (replaces NR's tag/workload model):

```
matcher: matchesValue(k8s.namespace.name, "*prod*")
field: environment
value: prod
```

Drop rules for cost optimization (decide which apply now vs. Wave 5):

- Drop health-check logs (`matchesValue(content, "*GET /health*")`)
- Drop DEBUG logs in production
- Sample sidecar proxy logs

6. Step Exit Criteria

S3 — Architecture Locked

- [ ] `bucket-design.md` final; bucket names + retention + pricing model documented
- [ ] `host-group-design.md` final; taxonomy understood by SRE + service owners
- [ ] `iam-model.md` final; security stakeholder signed off

- [ ] `openpipeline-design.md` final; routing rules sequenced
- [ ] Phase 1 Terraform module skeleton committed (will be applied in Wave 0 of Step 5)

Next step: NR2DT-04 — Translate (NRQL→DQL run; manual triage).

This notebook was AI-generated from community-submitted and publicly available sources, including the open-source NewRelic-to-Dynatrace-Migration-Utilities, nrql-engine, and nrql-translator projects. This notebook series is not officially supported by Dynatrace or New Relic. Always verify information against the official Dynatrace documentation and New Relic documentation.